

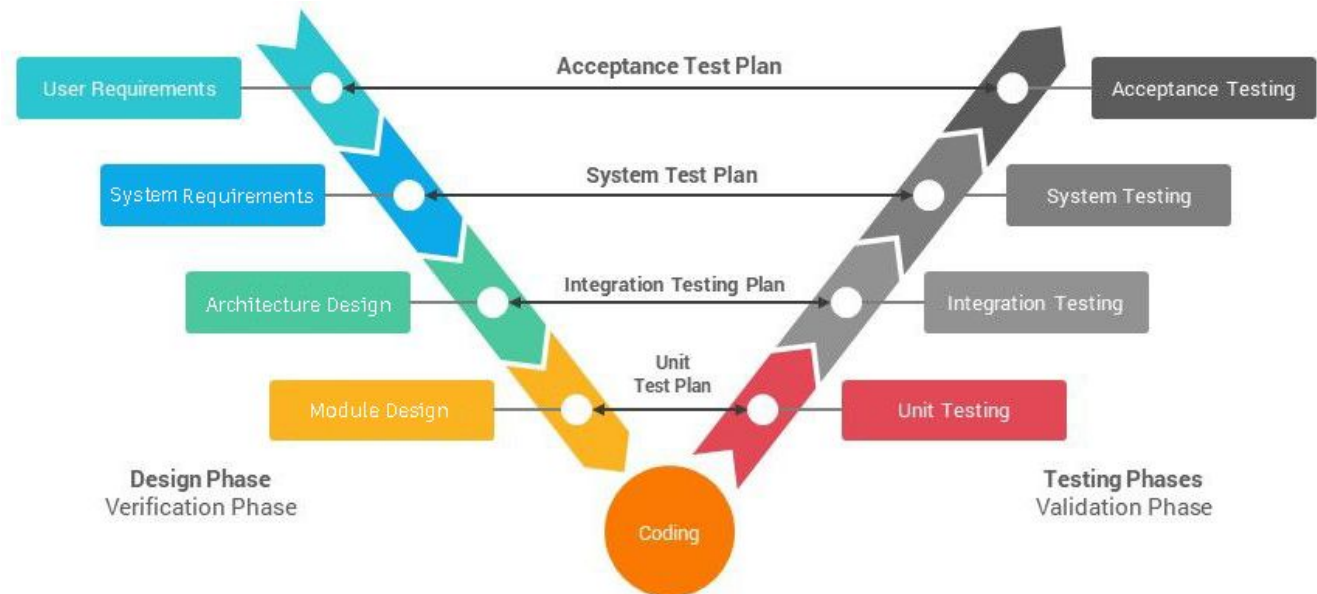


Implementation and Testing

Software Engineering

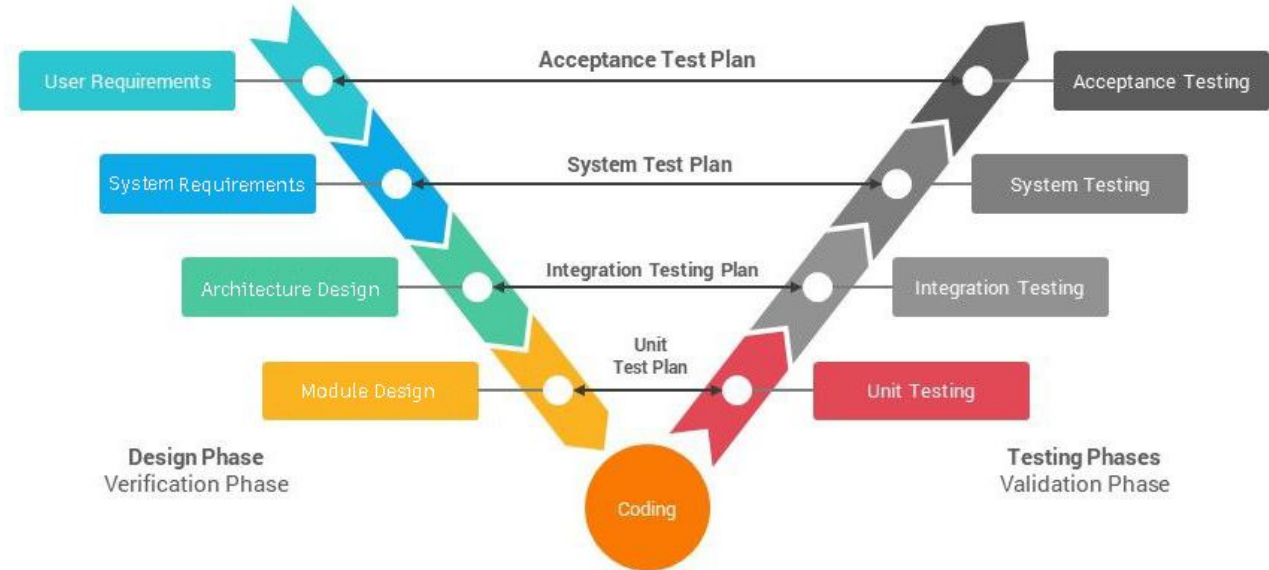
Implementation and Testing

- ❖ When it comes to the development and maintenance of high-quality software, comprehensive guidelines for the *implementation and testing phases* are used globally to ensure best practices in software engineering.
 - Software Life Cycle Processes
 - Test Documentation
 - Software Testing
 - Verification and Validation
- ❖ **Implementation Phase**
 - The implementation phase translates software designs into executable code.



Implementation Phase - Coding

- ❖ Implementation Phase includes
 - Coding
 - Integration
 - Implementation Verification



- ❖ **Coding** is the heart of the implementation phase and programmers transform detailed design into software code. This phase should follow coding standards to ensure maintainability, readability, and consistency across the codebase.

Implementation Phase - Coding

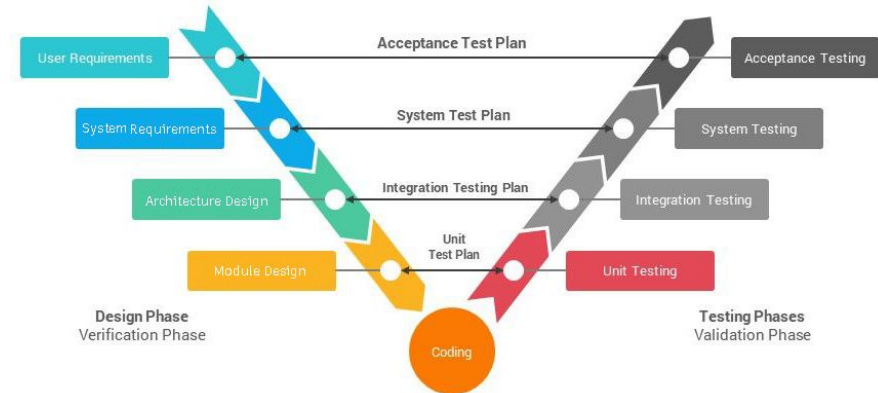
- ❖ The following practices are encouraged:
 - **Coding Standards:** Use organization-approved or project-specific coding standards to ensure uniformity. Standards define guidelines on naming conventions, indentation, commenting, and error handling.
 - **Documentation:** Code must be complemented by both inline comments and external documentation to enhance understanding and maintenance.
 - **Code Reviews/Inspections:** Regular peer reviews should be conducted to catch errors early and ensure the code follows best practices.
 - **Version Control:** Use version control systems (e.g., Git) to track changes in the code. This is essential for managing the evolving codebase during and after development.

Implementation Phase - Integration

- ❖ Once individual modules are developed, they must be integrated into the larger system. Integration can be performed incrementally (e.g., one module at a time)

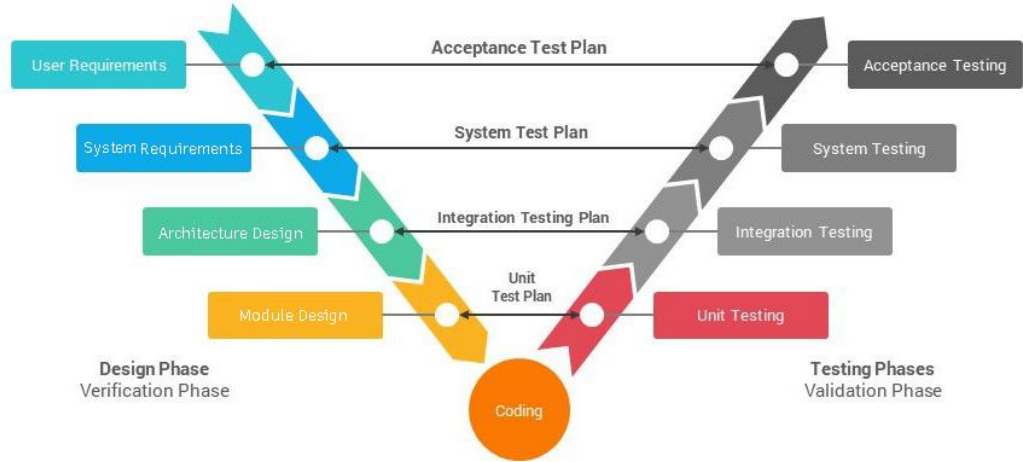
or as a "big bang" (all modules integrated simultaneously), although incremental integration is preferred because it allows for more manageable testing.

- **Integration Strategy:** An integration plan to outline how different modules will be combined and tested.
- **Continuous Integration:** Automated tools are used to ensure that as each module is integrated, tests are automatically executed to detect integration errors early.



Implementation Phase - Verification

- ❖ Ensure that the implemented software meets the design requirements through reviews and code inspections.
- ❖ **Static Code Analysis:** Before dynamic testing, static verification methods such as code walkthroughs or automated static analysis tools are used to detect issues like security vulnerabilities, dead code, and inefficiencies.
- ❖ **Unit Testing:** Each unit or module of the software is tested independently to verify it works as per its design specifications. Unit tests are typically written by developers and should cover both normal and boundary cases.

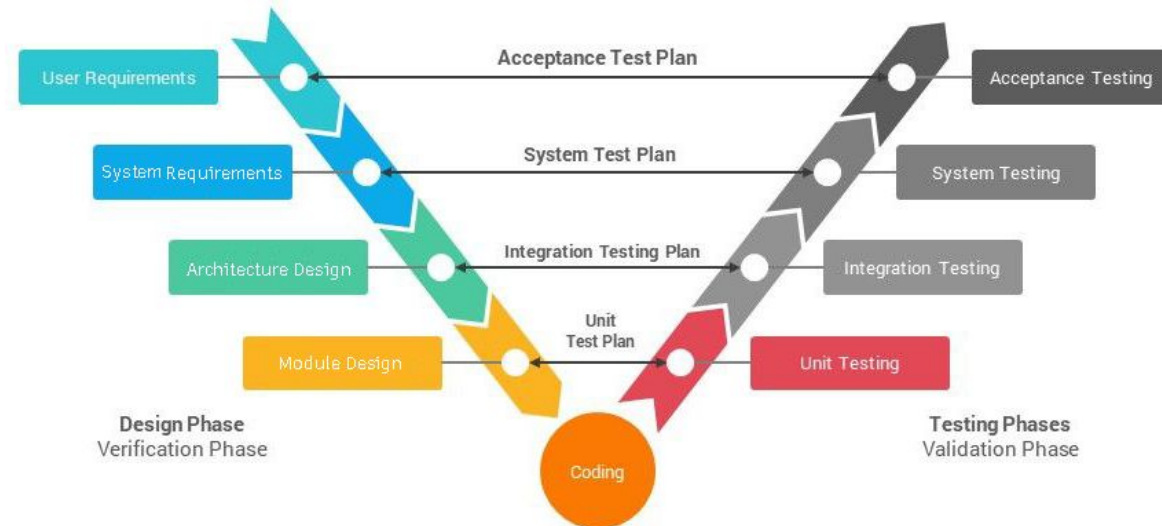


Testing Phase - Test Planning

❖ **Testing** is a part of the software life cycle

❖ The testing phase includes:

- Test Planning
- Test Design and Development
- Test Execution
- Test Reporting



❖ **Test Planning:** A detailed document outlining the approach, resources, schedule, and scope of testing. It includes the objectives, test items, features to be tested, responsibilities, and pass/fail criteria.

Testing Phase - Test Planning

- ❖ The *test plan document* includes the following sections:
 - **Test Items:** The list of software components that will be tested.
 - **Features to be Tested:** Specifies which software features will be tested.
 - **Approach:** Defines the testing strategies (e.g., functional, non-functional, regression testing) that will be applied, along with the test levels (unit, integration, system, acceptance).
 - **Entry and Exit Criteria:** Specifies the conditions under which testing can start and the criteria for determining when testing is complete.
 - **Resources:** Identifies the personal, tools, and other resources needed for testing.
 - **Schedule:** Provides a timeline for the testing activities.

Testing Phase - Test Design and Development

- ❖ **Test Design Specification:** This document describes the features of the software to be tested, along with the test cases and the test data required. It also maps test cases back to requirements, ensuring traceability.
- ❖ **Test Cases:** Each test case describes specific inputs, execution steps, and the expected results. Test cases are typically divided into:
 - **Functional Tests:** To verify the functional correctness based on requirements.
 - **Boundary Tests:** To ensure that the system behaves correctly at the limits of input ranges.
 - **Negative Tests:** To ensure the system handles invalid inputs gracefully.
 - **Performance Tests:** To measure the system's responsiveness and stability under load.
- ❖ **Test Data:** Test inputs and expected outputs for each case shall be clearly defined, whether they are manually entered or generated by test automation tools.

Testing Phase - Test Case Elements

- ❖ **Test Case Identifier:** A unique reference to distinguish it from other test cases.
- ❖ **Test Case Title:** A brief description of what the test case aims to verify.
- ❖ **Test Items:** Specifies what is being tested (modules, features, system elements).
- ❖ **Objective:** Describes the purpose of the test, linking it to specific requirements.
- ❖ **References:** Points to related documentation(requirement specifications) the test case covers.
- ❖ **Pre-conditions:** Conditions that must be met before the test can run.
- ❖ **Input Specifications:** Defines the data inputs required for the test.
- ❖ **Execution Steps:** A step-by-step process to execute the test.
- ❖ **Expected Results:** Describes what should happen if the test is successful.
- ❖ **Post-conditions:** Describes the state of the system after the test is complete.
- ❖ **Priority:** Indicates the importance of the test case (high, medium, low).
- ❖ **Test Execution Status:** Outcome of the test and details about who executed it and when.
- ❖ **Notes:** Additional information, such as environmental setup or special instructions.
- ❖ **Related Test Cases:** References to similar or related test cases.

Testing Phase - Test Case Example

❖ **Test Case ID:** TC_LOGIN_001

➤ A unique id for the test case, often based on the functionality being tested (e.g., login) and a number.

❖ **Test Case Title:** Verify Successful Login with Valid Credentials

❖ **Test Items:**

➤ System Under Test (SUT): Web-based application login module

➤ Module: User Authentication

❖ **Objective:** To verify that a user can successfully log into the application using valid credentials (username and password).

❖ **References:**

➤ Requirement ID: REQ_101 (Authentication) - Stakeholder Requirement ID

➤ Specification ID: SYS_201 (Login functionality specification) - System Requirement ID

Testing Phase - Test Case Example

❖ **Pre-conditions:**

- The user is already registered in the system.
- The login page is accessible.
- The browser is functional, and internet access is available.

❖ **Input Specifications:**

Username	testuser
Password	Pass@123

❖ **Execution Steps:**

- Open the web browser and navigate to the login page of the application.
- Enter "testuser" into the Username field.
- Enter "Pass@123" into the Password field.
- Click on the "Login" button.

Testing Phase - Test Case Example

❖ **Expected Results:**

- The user is successfully redirected to the homepage/dashboard.
- A welcome message with the user's name "Welcome, testuser" is displayed.
- The URL changes to <https://www.example.com/dashboard>.
- No error messages should be shown.
- The session is initialized and remains active as long as the user is interacting with the application.

❖ **Post-conditions:** The user is logged into the system to access functionalities associated with the user role.

❖ **Priority:** High

❖ **Test Execution Status:**

- Test Executed By: Eva Larson
- Date Executed: 2025-09-15
- Test Result: Passed (Example result, updated after execution)

Testing Phase - Test Case Example

❖ **Notes:**

- Ensure cookies and session storage are cleared before running this test case to avoid conflicts with previous sessions.

❖ **Related Test Cases:**

- TC_LOGIN_002: Verify login failure with incorrect password.
- TC_LOGIN_003: Verify login failure with empty fields.

❖ **Exercise:** Design a test case for your node in the greenhouse project.

Testing Phase - Test Execution

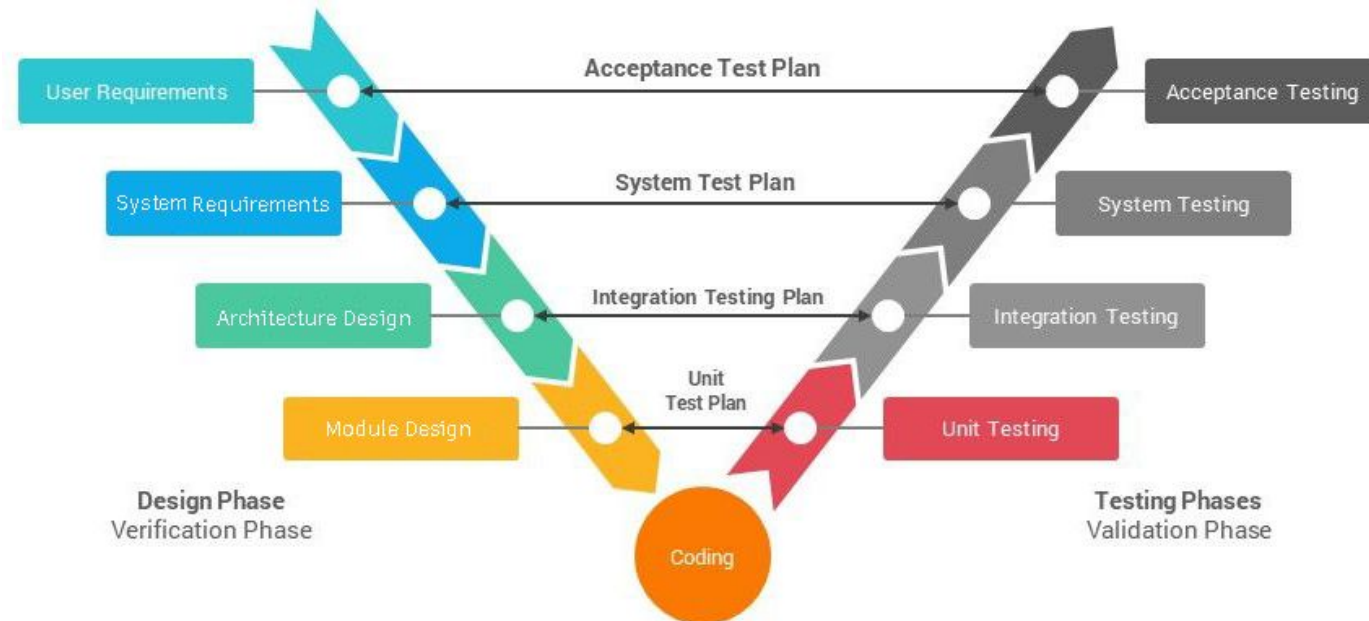
- ❖ **Test Procedure:** This defines the exact steps to be taken to execute each test case, including preconditions, inputs, the sequence of actions, and expected results.
- ❖ **Test Execution Log:** As tests are performed, results are recorded in the log, documenting whether each test passed or failed. If a test fails, the log should include a detailed description of the failure and any related defect reports.
- ❖ **Automated Testing:** Automated test scripts should be written and used to perform testing. Automation tools and frameworks are often used in conjunction with continuous integration systems.

Testing Phase - Test Reporting

- ❖ **Test Incident Report:** If a test case fails, an incident report must be generated. This includes a description of the issue, steps to reproduce, expected vs. actual behavior, severity, and priority.
- ❖ **Test Summary Report:** A comprehensive report summarizing all testing activities. The report includes key metrics such as:
 - **Test Coverage:** Percentage of software that was tested.
 - **Pass/Fail Rates:** Number of test cases that passed vs. those that failed.
 - **Defect Density:** Number of defects found per thousand lines of code (KLOC).
 - **Unresolved Defects:** A list of all unresolved or high-severity defects.
 - **Exit Criteria Compliance:** Confirmation that exit criteria were met (e.g., no critical defects).

Software Testing Methodologies - Test Levels

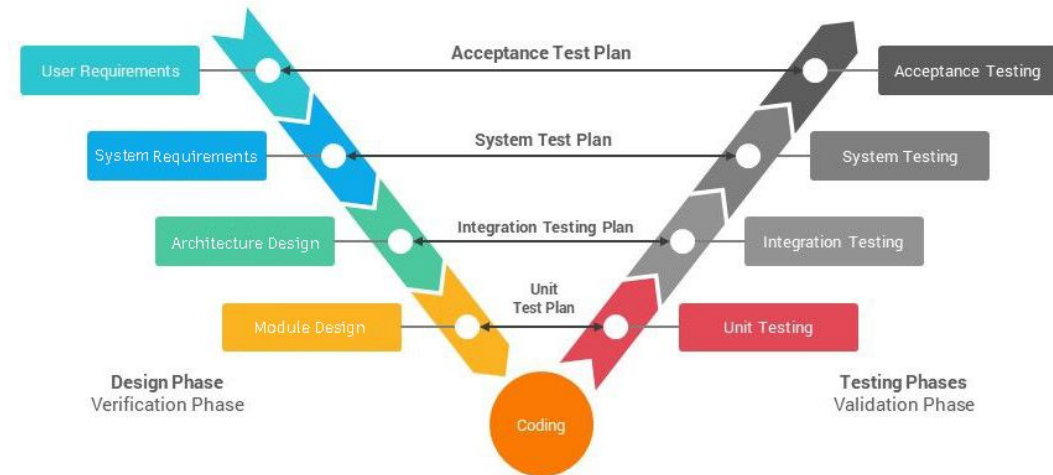
- ❖ **Test Levels:** define the different stages at which testing occurs, each focusing on different aspects of the software:
 - Unit Testing
 - Integration Testing
 - System Testing
 - Acceptance Testing



Software Testing Methodologies - Test Levels

❖ Unit Testing

- **Objective:** Test individual components or modules in isolation to verify that they function as expected.
- **Methodology:**
 - Typically performed by developers.
 - Test cases are written to verify that each function, method, or unit of code behaves correctly.
 - Focuses on validating internal logic, edge cases, and error handling.



Software Testing Methodologies - Test Levels

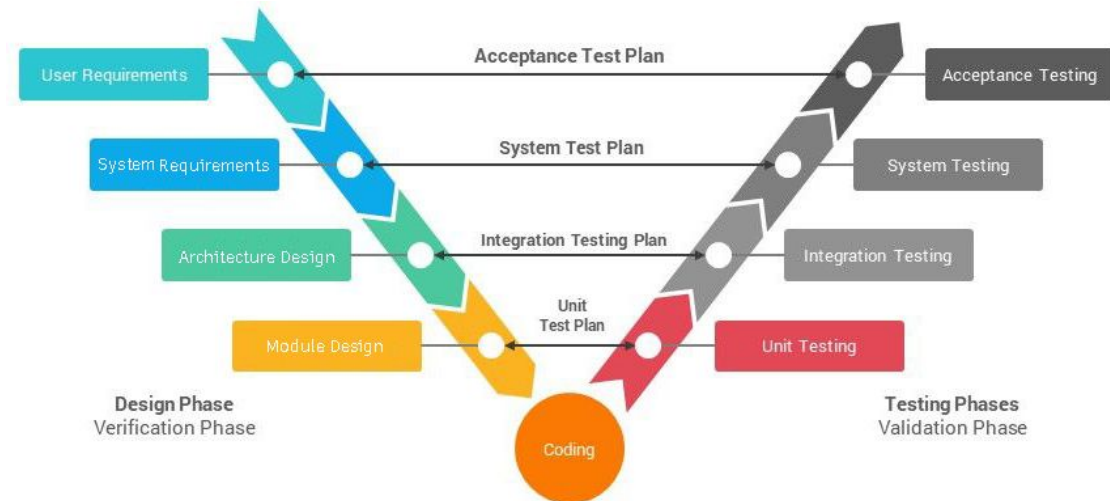
❖ Integration Testing

- **Objective:** Verify that different modules or components work together correctly.
- **Methodology:**
 - Focuses on the interaction between units or modules.
 - Conducted using a Top-down or Bottom-up approach, or a combination (called Sandwich Testing).
 - Incremental Integration Testing: Gradually integrates components one by one, ensuring that each new addition works in conjunction with previously integrated modules.
- **Types of Integration Testing:**
 - **Component Integration Testing:** Verifying interactions between individual software components.
 - **System Integration Testing:** Ensuring that the entire system integrates well with external systems or software.

Software Testing Methodologies - Test Levels

❖ System Testing

- **Objective:** Test the entire system as a whole to ensure that it meets the requirements.
- **Methodology:**
 - The fully integrated system is tested for compliance with functional and non-functional requirements.
 - Includes tests for functionality, performance, security, usability, and etc.
 - Executed in a test environment that closely resembles the production environment.



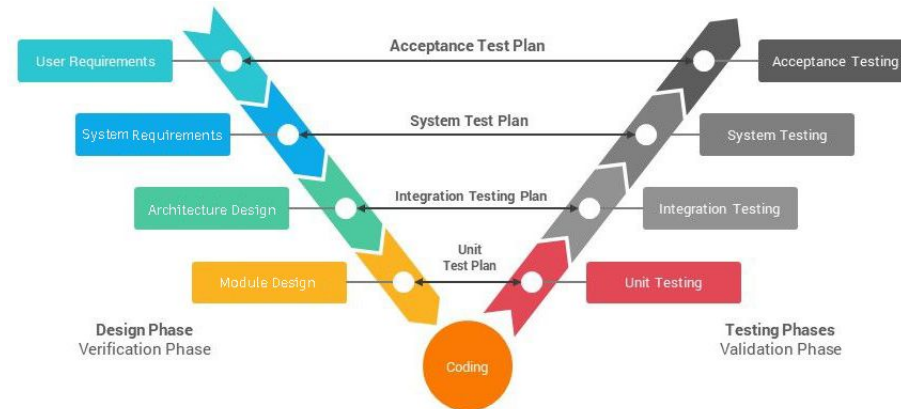
Software Testing Methodologies - Test Levels

❖ Acceptance Testing

➤ **Objective:** Ensure that the system meets the stakeholder requirements and is acceptable to the end-user or client.

➤ **Methodology:**

- Conducted by the client or users in an environment that simulates real-world usage.
- Tests if the system fulfills customer needs and contractual requirements.
- **Types of Acceptance Testing:**
 - **User Acceptance Testing (UAT):** End-users validate if the system meets their needs.
 - **Operational Acceptance Testing (OAT):** Validates operational readiness, such as backups, disaster recovery, and maintenance.



Software Testing Methodologies - Test Types

- ❖ Test types describe different methodologies to test specific aspects of the software.
 - Functional Testing
 - Non-Functional Testing
 - Black-Box Testing
 - White-Box Testing
 - Grey-Box Testing



Software Testing Methodologies - Test Types

❖ **Functional Testing**

- **Objective:** Verify that the software behaves according to the specified requirements.
- **Methodology:**
 - Based on black-box testing techniques, where the internal workings of the application are not considered.
 - Focuses on the software's functionality by inputting data and checking the output against the expected result.
 - Examples:
 - **Regression Testing:** Ensures that new code changes do not affect existing functionality.
 - **Smoke Testing:** To ensure that the critical functionality of the system is working.

Software Testing Methodologies - Test Types

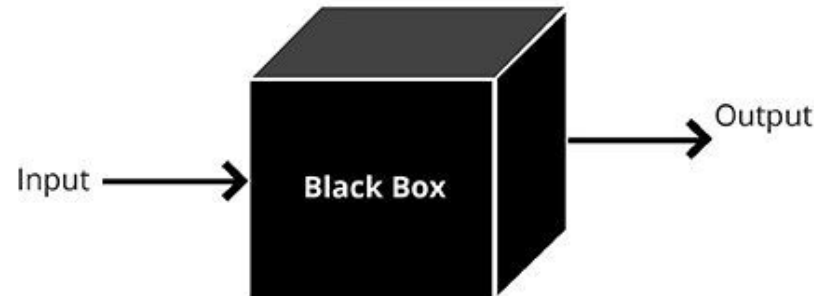
❖ Non-Functional Testing

- **Objective:** Assess non-functional aspects such as performance, usability, reliability, and security.
- **Methodology:**
 - Focuses on "**how**" the system performs rather than "**what**" the system does.
 - **Performance Testing:** Verifies the software's performance under different loads and conditions.
 - **Load Testing:** Checks how the system performs under expected load conditions.
 - **Stress Testing:** Puts the system under extreme load to identify its breaking point.
 - **Security Testing:** Ensures the software is free from vulnerabilities.
 - **Usability Testing:** Tests how easy it is for users to navigate and use the system.

Software Testing Methodologies - Test Types

❖ Black-Box Testing

- **Objective:** Test the functionality of the application without knowing the internal workings.
- **Methodology:**
 - Input values are provided, and outputs are observed without any knowledge of internal code.
 - Focuses on what the system should do, rather than how it does it.
 - Suitable for functional testing.
 - **Types:** Equivalence partitioning, Boundary value analysis, Decision table testing.



Software Testing Methodologies - Test Types

❖ White-Box Testing

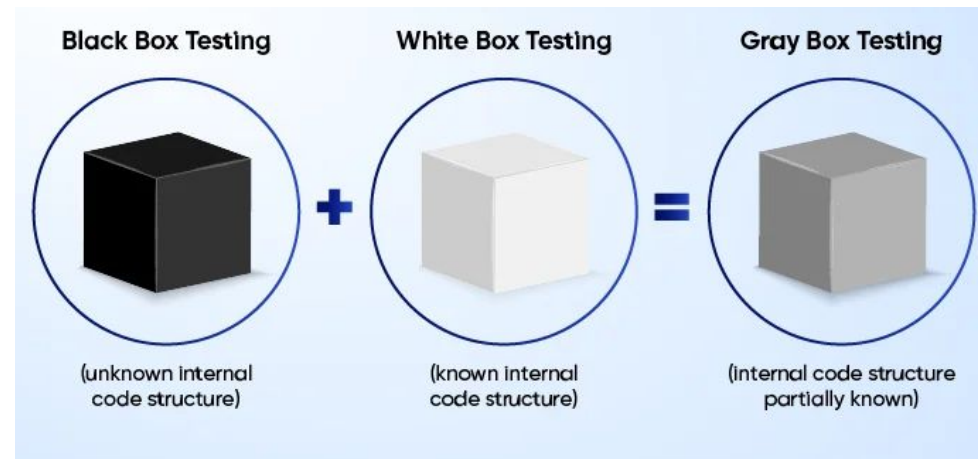
- **Objective:** Test the internal structure, logic, and code flow.
- **Methodology:**
 - The tester has full knowledge of the internal code.
 - Focuses on improving code quality and finding hidden bugs in code conditions, loops, etc.
 - Useful for unit and integration testing.
 - **Types:** Control flow testing, Data flow testing, Statement coverage, Branch coverage.



Software Testing Methodologies - Test Types

❖ Grey-Box Testing

- **Objective:** Combine both black-box and white-box testing techniques.
- **Methodology:**
 - The tester has partial knowledge of the system internals.
 - Useful in system testing and acceptance testing to simulate a real-world perspective while having some technical insight into the software structure.



Testing Methodologies

- ❖ There are several structured methodologies to guide how tests are conducted
 - Manual Testing
 - Automated Testing
 - Agile Testing
 - Risk-Based Testing



Testing Methodologies - Manual Testing

- ❖ **Objective:** Testing conducted manually by testers to find bugs or issues without help of automation tools.
- ❖ **Methodology:**
 - Involves creating detailed test cases, executing them manually, and recording the results.
 - Ideal for human-driven testing like usability testing, exploring the application freely, or doing quick, unscripted checks.
- ❖ **Test Documentation:**
 - **Test Case Specification:** Describes the test cases to be executed.
 - **Test Procedure Specification:** Specifies the steps to execute each test case.
 - **Test Log:** A record of all test cases executed and their outcomes.

Testing Methodologies - Automated Testing

- ❖ **Objective:** Utilize automated tools to run tests, compare actual outcomes with expected results, and report issues.
- ❖ **Methodology:**
 - Suitable for repetitive tests such as regression tests, performance tests, and stress tests.
 - Saves time and effort on large projects where manual testing is impractical.
- ❖ **Guidelines:**
 - Automated tests should be well documented and integrated into the overall test plan.
 - Automated test scripts must be version-controlled and maintained alongside the software.

Testing Methodologies - Agile Testing

- ❖ **Objective:** In agile methodologies, testing is conducted continuously and in parallel with development.
- ❖ **Methodology:**
 - Testing is integrated into sprints with continuous feedback loops.
 - Test-Driven Development (TDD): Tests are written before code is implemented.
 - Behavior-Driven Development (BDD): Focuses on testing the behavior of an application in user-friendly language.
- ❖ **Alignment:**
 - Guidelines emphasize the importance of continuous testing and early defect detection, aligning well with agile principles.
 - Testing documentation in agile projects may be lighter but still necessary, focusing on traceability and coverage.

Testing Methodologies - Risk-Based Testing

- ❖ **Objective:** Focuses testing efforts on areas of the software that are most critical or prone to failure.
- ❖ **Methodology:**
 - Tests are prioritized based on risk analysis, with higher risk areas being tested first.
 - Risk is assessed based on factors such as business impact, code complexity, and potential for defects.
- ❖ **Guidelines:**
 - Encourages risk assessment during the planning phase to guide test case selection and resource allocation.
 - Risk-based testing ensures that the most critical components receive enough test coverage, even in time-constrained situations.

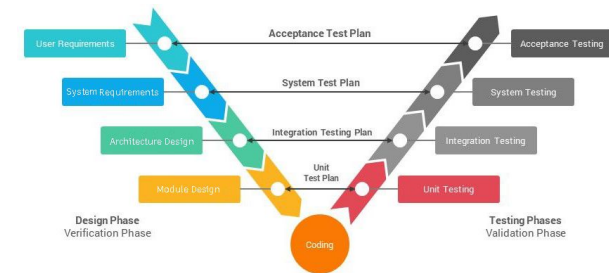
Testing Methodologies - Verification and Validation

❖ Verification

- **Objective:** Ensure that the product is built correctly according to its design/specifications.
- **Methodology:**
 - Focuses on internal consistency, correctness, and completeness.
 - Involves code reviews, inspections, walkthroughs and static analysis
 - Are we building the product right?

❖ Validation

- **Objective:** Ensure that the right product is built to meet user needs.
- **Methodology:**
 - Testing the software in real-world or simulated environments.
 - Ensures that the software meets customer expectations.
 - Are we building the right product?



Implementation and Testing - Key Considerations

- ❖ **Documentation:** Comprehensive documentation is crucial in all stages of implementation and testing.
- ❖ **Traceability:** Ensuring that requirements are linked to test cases.
- ❖ **Iterative Testing:** Testing should be continuous and iterative throughout the development cycle.
- ❖ **Independent Testing:** To minimize bias, independent testing teams are recommended.

Implementation and Testing

❖ Some useful links

- [How To Keep Development Teams Running](#)
- [Testing Phase in SDLC: Ensuring Quality and Reliability](#)
- [7 Phases of the System Development Life Cycle Guide](#)
- [SDLC in Software Testing: Phases, Benefits, Models & Types](#)
- [Software Testing Methodologies](#)
- [Software Testing Methodologies: Each Explained](#)
- [Software Testing Methodology - A Complete Overview](#)
- [Software Testing Methodologies Guide: A High-Level Overview](#)
- [6 QA testing methodologies and techniques in 2024](#)
- [Software Testing Techniques](#)